EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 4-BN: Binary Ninja Personal — Resolve, Hack, and Patch the RP2350 (Raw `.bin`)

ARM Cortex-M33 • Binary Ninja • OpenOCD

October 12, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the two lesson projects with `Release` and get both an `.elf` and a raw `.bin`
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load the raw `.bin` into Binary Ninja at `0x10000000`
- **Break at** `main` on live silicon, even though `main` can move between programs
- **Hack a running target live** by editing a register in Binary Ninja's Registers widget
- **Resolve the functions in the Binary Ninja GUI** using the ELF symbol map
- **Patch** the bytes that control the behavior, export the image, and flash it

How This Guide Works

The build produces two files for each project:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table	Ground truth for every function address and name
`.bin`	The raw flash image, no headers, no symbols	The image we load into Binary Ninja and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in Binary Ninja, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the Week 4 lesson, and the Week 4 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it folds `age = 42` away in Project 1 and inlines `blink_and_print` into `main` in Project 2. If you build `Debug`, the SDK function addresses move and Project 2 keeps a separate `blink_and_print`, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**, twice — once per project:

1. Break on the live target and prove what the code does.
2. Hack it live in the debugger and watch the behavior change.
3. Resolve the functions in Binary Ninja using the ELF symbol map.
4. Patch the bytes, export, convert, and flash.

Project	Prints	Also does	The hack
`0x0005_intro-to-variables`	`age: 43`	loops on `printf`	change `43` to `70`
`0x0008_uninitialized-variables`	`age: 0`	blinks the red LED on GPIO 16	change `0` to `66`, move the LED to GPIO 17

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

Part 1: Build, Flash, and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **Binary Ninja Personal** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

```
brew install cmake ninja
```

- Install **Binary Ninja Personal** and complete its license activation.
- Install the **Arm GNU Toolchain**, or let the VS Code Pico extension manage it.

Linux x64

```
sudo apt install cmake ninja-build gcc-arm-none-eabi libnewlib-arm-none-eabi git python3 openocd
minicom
```

- Install **Binary Ninja Personal** and complete its license activation.

Step 2: Verify your tools are the right architecture (do not skip this)

On **macOS Apple Silicon**, the most common failure is an Intel `x86_64` tool on your `PATH`:

```
zsh: bad CPU type in executable: cmake
```

You may have **two Homebrews**: the arm64 one at `/opt/homebrew` and the Intel one at `/usr/local`. If `/usr/local/bin` wins, every `brew` tool is x86_64. Check:

```
file "$(which cmake)"
file "$(which ninja)"
file "$(which arm-none-eabi-gdb)"
file "$(which arm-none-eabi-nm)"
file "$(which openocd)"
file "$(which telnet)"
```

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

To make it permanent, add that `export` to `~/.zshrc`. Do not use Rosetta as a fix; OpenOCD and GDB are exactly the kind of programs where a translation layer produces failures that look like debugger bugs.

`telnet` **is special — and optional.** The GDB MI workflow does not need it; it is only used by the command-port fallback. macOS no longer ships `telnet`, and the Homebrew build is often the Intel one, so `telnet 127.0.0.1 4444` fails with `bad CPU type in executable`. Your `brew` command itself may also be the Intel one: if `brew install telnet` fails with `.../portable-ruby/.../ruby: Bad CPU type in executable`, you are running the Intel Homebrew. Call the Apple Silicon Homebrew explicitly:

```
/opt/homebrew/bin/brew install telnet
```

If you would rather not install anything, macOS ships an arm64 `nc`, which can connect to the same OpenOCD port:

```
nc 127.0.0.1 4444
```

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the two projects with `Release`

Run this once inside `0x0005_intro-to-variables/` and once inside `0x0008_uninitialized-variables/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point Binary Ninja at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so Binary Ninja never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the Binary Ninja console, so the whole build -> patch -> flash loop stays inside Binary Ninja. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
for name in ("0x0005_intro-to-variables", "0x0008_uninitialized-variables"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
for name in ("0x0005_intro-to-variables", "0x0008_uninitialized-variables"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
for name in ("0x0005_intro-to-variables", "0x0008_uninitialized-variables"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Each build directory now contains the pair we need:

- `0x0005_intro-to-variables/build/0x0005_intro-to-variables.elf` and `.bin` — `.bin` is **15292** bytes
- `0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.elf` and `.bin` — `.bin` is **15668** bytes

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin`
Linux x64	`/usr`

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on each ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x0005_intro-to-variables.elf | grep -E ' [Tt] '
arm-none-eabi-nm -n --defined-only build/0x0008_uninitialized-variables.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x0005_intro-to-variables.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T` / `t` type is a function. Here are the functions this lesson uses.

Project 1 — `0x0005_intro-to-variables`:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function
`0x10000248`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO helper
`0x10002cfc`	`exit`	`void exit(int)`	C runtime exit
`0x10002d04`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10002f54`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x100030e4`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper

These signatures come from the ELF's DWARF debug info, so they are exact. You apply them in Binary Ninja in Step 16 (`G` -> address, then `Y` -> Change Type).

Project 2 — `0x0008_uninitialized-variables`:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`

Address	ELF symbol	Signature	Role
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (`blink_and_print` inlined)
`0x10000278`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO helper
`0x100002b4`	`gpio_init`	`void gpio_init(uint)`	SDK GPIO init
`0x10000d10`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK delay
`0x10002e74`	`exit`	`void exit(int)`	C runtime exit
`0x10002e7c`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x100030cc`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x1000325c`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper

> **`main` is `0x10000234` in both projects.** In Project 2 the `static void blink_and_print` helper is inlined into `main` by the `Release` optimizer, so it does not appear as a separate symbol. That is why both projects put `main` at the same address. In a `Debug` build it stays separate and `main` moves — another reason to build `Release`.

Step 5: Flash Project 1 and confirm `age: 43`

A `.bin` has no headers, so OpenOCD must be told the base address `0x10000000`. From the repository root:

macOS Apple Silicon / Linux x64:

```
./flash.sh 0x0005_intro-to-variables/build/0x0005_intro-to-variables.bin
```

Windows x64 (PowerShell):

```
.\flash.ps1 -Bin 0x0005_intro-to-variables\build\0x0005_intro-to-variables.bin
```

Or flash from the Binary Ninja console (with a database open, so the repo root is taken from it — otherwise use the terminal form above):

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0005_intro-to-variables", "build", "0x0005_intro-to-
variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0005_intro-to-variables", "build", "0x0005_intro-to-
variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 15292 bytes ...` and `** Verified OK **`. Open a serial monitor at **115200** baud:

- **Windows x64:** PuTTY -> Connection type **Serial**, the Pico's COM port, speed `115200`.
- **macOS Apple Silicon:** `screen /dev/tty.usbmodem* 115200` (quit with `Ctrl-A` then `K`).
- **Linux x64:** `minicom -D /dev/ttyACM0 -b 115200`.

```
age: 43
age: 43
age: 43
...
```

Step 6: Flash Project 2 and confirm `age: 0` + red LED

```
# macOS / Linux
./flash.sh 0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin
```

```
# Windows
.\flash.ps1 -Bin 0x0008_uninitialized-variables\build\0x0008_uninitialized-variables.bin
```

Or flash from the Binary Ninja console (same form as Step 5, pointing at the Project 2 `.bin`):

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0008_uninitialized-variables", "build", "0x0008_uninitialized-
variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0008_uninitialized-variables", "build", "0x0008_uninitialized-
variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 15668 bytes ...`. The serial monitor shows:

```
age: 0
age: 0
age: 0
...
```

and the **red LED on GPIO 16** blinks once per second.

Part 2: Load the Raw `.bin` into Binary Ninja

Start from a fresh Binary Ninja state. If you already have a `.bndb` for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into Binary Ninja

A raw `.bin` has no headers, so Binary Ninja cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, Binary Ninja may load it at address `0x0` with a guessed architecture, and every address in this lesson will be wrong.

1. Choose `File -> Open with Options...` (do **not** use plain `File -> Open`).
2. Select `0x0005_intro-to-variables/build/0x0005_intro-to-variables.bin`.
3. In the loader options, set:
   - **Architecture:** `thumb2` (the ARMv7-M / ARMv8-M Thumb-2 architecture, which covers the Cortex-M33)
   - **Platform:** `thumb2`
   - **Base Address:** `0x10000000` (the XIP flash base)
4. Click **Open**.

Binary Ninja analyzes the image and opens the linear view.

Verify the load before going further. Press `G`, type `0x10000000`, and read the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you instead see data at `0x00000000`, or a vector word without bit 0 set, close the tab and repeat with `Open with Options`. The Cortex-M33 only executes Thumb-2, so `thumb2` is the only correct architecture.

> **Console equivalent:**
>
> ```
> load("0x0005_intro-to-variables/build/0x0005_intro-to-variables.bin",
>     options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Step 8: Save it as a Binary Ninja database (`.bndb`)

Binary Ninja never writes back into the `.bin`. Your names, comments, types, and patches live in a separate `.bndb` database. Save one now, before you make any changes:

1. Choose `File -> Save As...`.
2. Save it next to the image as `0x0005_intro-to-variables.bndb`.
3. From now on, save with `File -> Save` (`Cmd+S` on macOS, `Ctrl+S` on Windows/Linux) whenever you rename or patch.

The two files have different roles:

File	Role
`0x0005_intro-to-variables.bin`	the raw firmware image; Binary Ninja never modifies it
`0x0005_intro-to-variables.bndb`	your analysis database: names, types, comments, and patches

When you come back later, **open the** `.bndb`, not the `.bin`; that restores all your work. If a database gets messy, delete the `.bndb` and re-import the `.bin` from Step 7 — the firmware is never at risk. You export the patched image out of this view later, in Step 19.

Step 9: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment. Breakpoints are set from the OpenOCD command port, not from the GUI — see Step 13.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Dynamic — Break at `main` and Hack Live (Project 1)

Step 10: Start OpenOCD as a live debug server

Make sure no other OpenOCD is running; a forgotten server holds port `3333`.

macOS / Linux:

```
ps aux | grep -i openocd
```

Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -like '*openocd*' }
```

Stop any leftover server gracefully:

macOS / Linux:

```
pkill -TERM -f openocd
```

Windows (PowerShell):

```
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Start the server **parked at** `main`:

macOS Apple Silicon / Linux x64:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Windows x64 (PowerShell):

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Or start it from the Binary Ninja console, freeing the probe first and launching the server in the background so the console returns immediately:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                      os.path.join(root, "debug-server.ps1")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

`Popen` returns in a few milliseconds; the server keeps running in the background. Check `openocd.log` for `Listening on port 3333`, then connect in Step 11.

Wait for:

```
Info : [rp2350.dap.core0] Examination succeed
Startup breakpoint at 0x10000234 (2-byte hardware execute, one-shot).
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

> `BP_ADDR` **parks the core at** `main` **before any client connects.** The script arms a 2-byte hardware breakpoint and then does the startup `reset run`, so the core runs from the vector table and stops at your address with no debugger attached yet. When Binary Ninja connects a moment later, the first thing it reads is already the truth: `Stopped at 0x10000234`. This is the whole reason the lab works cleanly — you never have to drive a reset from outside the GUI.
>
> Use the address you actually want to stop at:
>
> | What you want to stop at | Project 1 `0x0005` | Project 2 `0x0008` | Command |
> |---|---|---|---|
> | `main` (once per reset) | `0x10000234` | `0x10000234` | `BP_ADDR=0x10000234 ./debug-server.sh` |
> | The **loop** — the `printf` call, hit every iteration | `0x1000023e` | `0x1000024e` | `BP_ADDR=0x1000023e ./debug-server.sh` |
>
> ```
> BP_ADDR=0x10000234 ./debug-server.sh   # park at main
> BP_ADDR=0x1000023e ./debug-server.sh   # park in the loop instead
> ```
>
> ```
> $env:BP_ADDR="0x10000234"; .\debug-server.ps1   # park at main
> $env:BP_ADDR="0x1000023e"; .\debug-server.ps1   # park in the loop
> ```
>
> **Note the loop address is not the same in both projects.** Project 2 does more setup before the loop, so its `bl __wrap_printf` sits at `0x1000024e`, not `0x1000023e`. Both were verified against the Release `.elf` with `arm-none-eabi-objdump` and confirmed live on hardware.
>
> **This startup stop is single-use.** OpenOCD flushes breakpoints when a client connects, so this one is gone once Binary Ninja attaches — fine for `main`, which only runs once per reset. Every breakpoint after that is set from the Binary Ninja GUI (Step 13) and is repeatable. To stop at `main` again, restart the server with `BP_ADDR` and reconnect.

> **Exactly one core.** The line must say `core0` and must **not** mention `core1`. Core1 is never started by this firmware; exposing it makes Binary Ninja read core1's reset-state registers, which are not real addresses, and OpenOCD floods the log with `Failed to read memory at 0xf0000000`. The scripts already use `USE_CORE=0`; do not change it.

> **Windows driver note:** the Debug Probe must use the **WinUSB** driver. If OpenOCD reports `unable to open CMSIS-DAP device`, install it with Zadig (select `Debug Probe (CMSIS-DAP)` -> WinUSB).

Step 11: Connect Binary Ninja to the GDB server

1. Make sure the image is open and analyzed (Part 2) and the server from Step 10 is running (parked at `main`).
2. Choose `Debugger -> Connect to Remote Process`.
3. In the **adapter** dropdown, select **GDB MI**.
4. In the **connect** settings group, set **IP Address** to `127.0.0.1` and **Port** to `3333`.
5. Set **Full GDB Executable Path** to the `arm-none-eabi-gdb` from the **Arm GNU Toolchain 14.2.rel1**. It ships for all three hosts, and the Raspberry Pi Pico VS Code extension installs that same 14.2.rel1 toolchain (including `arm-none-eabi-gdb`) on all of them:

OS	`arm-none-eabi-gdb` path
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb` (or the Pico extension's `~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb`)
Windows x64	`%USERPROFILE%\.pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe` (Pico extension), or `C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.2 rel1\bin\arm-none-eabi-gdb.exe`
Linux x64	`~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb` (Pico extension), or the `bin/` directory of the extracted `arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi` tarball

6. Click **Accept**.
7. Restrict the GDB memory map (prevents OpenOCD SWD faults and reconnect loops): In the bottom console panel, click the **Debugger Console** tab (or `View -> Tabs -> Debugger Console`) and enter:

```
source pico-gdb-memory.gdb
```

(If not running from the repository root, enter the absolute path to `pico-gdb-memory.gdb`*.)* Confirm `info mem` displays the Flash (`0x10000000 0x10400000 ro`) and SRAM (`0x20000000 0x20082000 rw`) ranges.

> **Why `source pico-gdb-memory.gdb` is required.** Binary Ninja's register widget automatically previews memory pointed to by registers. When a register contains an unmapped address (such as uninitialized data or constants like `0x69000000`), Binary Ninja probes it. Without a bounded memory map, OpenOCD attempts to read that invalid address over SWD, faults, and reconnects the debug port (`Info : SWD DPIDR 0x4c013477`). Sourcing `pico-gdb-memory.gdb` sets `inaccessible-by-default on` in GDB, causing GDB to intercept and reject invalid reads locally before any packet reaches OpenOCD, eliminating SWD reconnect storms.

> **Use the GDB MI adapter.** It launches a real `arm-none-eabi-gdb --interpreter=mi2` and lets Binary Ninja drive it, so breakpoints and stepping go through real GDB — which sends the correct 2-byte breakpoint length and handles step-over itself. Verified working end to end: connect, GUI breakpoints (**Add Hardware Breakpoint...**, hardware execute), **Step Into** / **Step Over**, and register edits. Stops are reported as `Breakpoint` (not `SingleStep`).
>
> **Do NOT have any breakpoints set in Binary Ninja before you connect.** With the GDB MI adapter, attaching while Binary Ninja already has a breakpoint **hangs the session**. Start the server parked with `BP_ADDR` (Step 10), connect, and only add hardware breakpoints *after* the connection is up. This is a Binary Ninja bug; it is the single most common GDB MI failure.
>
> **The GDB executable path matters.** Use the **14.2.rel1** build on every OS (Windows, macOS, Linux). The 13.3.rel1 build — which is what `/opt/homebrew/bin/arm-none-eabi-gdb` symlinks to — did **not** connect in testing.
>
> **This step is temporary.** Vector35 plans to ship a GDB binary with the GDB MI adapter (Vector35/debugger#929, milestone *Langara*). Once that lands, Binary Ninja provides GDB itself and you will not need to set **Full GDB Executable Path** at all.
>
> **Do not pick Corellium.** Binary Ninja's adapter dropdown also lists **Corellium**, which is for Corellium's virtual devices and expects an API token, not a local OpenOCD server. It is not the adapter for this lab. The dropdown is a combo box, so an accidental arrow-key press can land on it — always read the label back and confirm it says **GDB MI** before clicking **Accept**.

> **The adapter and port are not saved in the `.bndb`.** Every time you relaunch Binary Ninja you must re-select **GDB MI**, re-enter port `3333`, and re-set the GDB path.

> **Watch for an off-screen error dialog.** When a connection fails, Binary Ninja pops a `Binary Ninja critical alert` window that can be positioned mostly outside the main window (seen at `2430,331` with the main window at `2560,30`), which makes it look like nothing happened. If the connect seems to do nothing, check your other display.

The target keeps running. Open the **Registers** tab (bug icon) and confirm you see live values. `pc` inside `0x10003xxx` and `sp` just below `0x20082000` are healthy.

> **If `pc` is `0x00000088`, `0x000000ec`, or `sp` is `0xf0000000`, the session is bad.** Restart the server, then restart Binary Ninja (a server restart while attached leaves Binary Ninja in a stale session), and connect again.

Step 12: Find `main` without relying on its address

`main` can move between programs, so we do not guess it. We follow the one fixed path to it. Press `G` and go to `0x10000000`:

```
0x10000000   0x20082000   initial stack pointer (top of SRAM)
0x10000004   0x1000015d   reset vector
```

Bit 0 of a vector is the Thumb bit, so `0x1000015d` means "start at `0x1000015c`". That is `_reset_handler`. Follow the reset path to `0x10000186`, `platform_entry`:

```
10000186 <platform_entry>:
10000186: 4914   ldr  r1, [pc, #80] ; @ 0x100001d8
10000188: 4788   blx  r1            ; runtime_init
1000018a: 4914   ldr  r1, [pc, #80] ; @ 0x100001dc
1000018c: 4788   blx  r1            ; main  <-- the fixed anchor
1000018e: 4914   ldr  r1, [pc, #80] ; @ 0x100001e0
10000190: 4788   blx  r1            ; exit
10000192: be00   bkpt 0x0000
```

The middle `blx` at `0x1000018c` is the call to `main`. `platform_entry` is byte-identical in both projects, so `0x1000018c` catches `main` no matter where the linker placed it. The literal pool at `0x100001dc` holds `main | 1`, and clearing bit 0 gives `0x10000234`.

Step 13: Set a hardware breakpoint in the GUI

With the **GDB MI** adapter, Binary Ninja sets breakpoints through real GDB, which sends the correct 2-byte length, so you set them **in the UI**. There is no command port here.

> **Why older drafts used the command port.** Binary Ninja's **GDB RSP** adapter is its own minimal RSP client and sends a **1-byte** breakpoint (`Z0,<addr>,1`); the Cortex-M33 FPB comparators need 2 bytes, so OpenOCD rejected it with `only breakpoints of two bytes length supported`. The old workaround was to arm breakpoints by hand over telnet. **The GDB MI adapter does not have this problem** — it drives real `arm-none-eabi-gdb`, which sends the right length. So everything below is done in the GUI. The command port still exists as a fallback (see the end of this step), but you do not need it.

Where you can stop

You want to stop at	Address	How	Repeatable?
`main`	`0x10000234`	The server starts parked there with `BP_ADDR=0x10000234` (Step 10), so Binary Ninja is already stopped at `main` when it connects.	No — `main` runs once per reset.
The loop (`printf` call)	Project 1 `0x1000023e`, Project 2 `0x1000024e`	Set a hardware breakpoint in the GUI, then click **Resume**.	Yes — fires on every iteration.

Set the loop breakpoint in the GUI

1. Press `G`, type the loop address (`0x1000023e` for Project 1, `0x1000024e` for Project 2), and press Enter.
2. Set a **hardware execution** breakpoint at that address, either way:
   - `Debugger -> Add Hardware Breakpoint...` — a **hardware execute** (`HE`) breakpoint. **Use this one.**
   - click the line and press `F2` (`Debugger -> Toggle Breakpoint`) — a **software** breakpoint. It will **not** work here: the code is in read-only flash, so GDB cannot install it and the core just keeps running.
3. Click **Resume**. The core is already running the loop, so the breakpoint fires on the next iteration. Binary Ninja stops with the PC at the loop address and reports it as a **Breakpoint** — verified: `Stopped (Breakpoint) at 0x1000023e`.

> **No breakpoints before you connect.** With GDB MI, a breakpoint set before the connection hangs the session (Step 11). Start parked with `BP_ADDR`, connect, *then* add breakpoints.

Stepping

With the target halted at the breakpoint, **Step Into** (`F7`) and **Step Over** (`F8`) run through real GDB and move the PC. Verified: `0x1000023e -> 0x100030e4 -> 0x100030e6 -> ...` .

> **Step Over on the raw `.bin` steps *into* calls.** The raw image has no symbol for `__wrap_printf` , so **Step Over** at the `printf` call behaves like **Step Into**. When the lab needs to execute the call and then stop, it moves the breakpoint to the return site and clicks **Resume** instead (Step 14 shows this).

> **Never use Binary Ninja's Restart button.** On RP2350 it resets and halts inside the boot ROM (`pc=0x88` , `sp=0xf0000000`). To reset cleanly, restart the server with `BP_ADDR` and reconnect.

> **If you ever need the command port.** It is still there — `nc 127.0.0.1 4444` , and `bp <addr> 2 hw` still arms a breakpoint, `rbp <addr>` / `rbp all` still remove them. It is the fallback if you switch back to the **GDB RSP** adapter, whose 1-byte breakpoints the GUI cannot set. With GDB MI you do not need it for this lab.

Step 14: HACK IT LIVE — change the printed value

`main` loads the constant `0x2b` (43) into `r1` and calls `printf` on every iteration. We break on that call in the GUI and change it live.

1. Press `G` , go to `0x1000023e` (the `bl __wrap_printf`).
2. Set a **hardware execute** breakpoint there: `Debugger -> Add Hardware Breakpoint...` . (Do not use `F2` — that is a software breakpoint and will not work on read-only flash.)
3. Click **Resume** in Binary Ninja. The target is already running the loop, so the breakpoint fires on the next iteration. Binary Ninja stops with the program counter at `0x1000023e` and `r1 = 0x2b` .
4. Open the **Registers** widget (bug icon -> **Registers**).
5. Find `r1` . Its value is `0x2b` .
6. **Set `r1` to `0x46` .** From Binary Ninja's Python console (`Plugins -> Python Console`):

   ```
   dbg.set_reg_value("r1", 0x46)
   ```

   `dbg.set_reg_value(name, value)` writes one register (returns `True` on success). You can also right-click `r1` in the **Registers** widget, press `E` (edit), type `46` , and press Enter. The widget may not repaint the value, but the write reaches the target — you confirm it by the printed output in the next steps.
7. **Move the breakpoint past the call.** You want `printf` to run once and then stop, so move the breakpoint from `0x1000023e` to the instruction *after* the call, `0x10000242` (the `b.n` that closes the loop): remove the breakpoint at `0x1000023e` and set a hardware breakpoint at `0x10000242` . Two reasons not to just click **Step Over** here: a breakpoint left on the current PC re-traps the step, and Binary Ninja's **Step Over** steps *into* `__wrap_printf` on this raw `.bin` because the image carries no symbol for the call. Moving the breakpoint to the return site is deterministic.
8. Click **Resume** in Binary Ninja. The core executes `bl __wrap_printf` with `r1 = 0x46` , so this iteration prints `age: 70` , then stops at `0x10000242` .
9. Look at your serial monitor — the `screen` session on the Pico's USB serial port — and at the **Target** tab in Binary Ninja:

   ```
   age: 70
   ```

You changed a running program's output without touching the binary.

Step 14b: HACK THE STRING LIVE — change `age:` to `foo:`

The text `"age: %d\r\n"` lives in flash (`.rodata`) at `0x100034a0` , and flash is **read-only at runtime** — a debugger write there does not stick (verified: writing `0x66` to `0x100034a0` read back unchanged). So you cannot overwrite the text in place. Instead you redirect the pointer: at the `printf` call, `r0` holds the string address, so you point `r0` at a replacement string you place in RAM.

1. Arm the breakpoint at the `printf` call and hit it, exactly as in Step 14 steps 1-3. At the stop, `r0 = 0x100034a0` and `r1 = 0x2b` .
2. Put the replacement string into free RAM at `0x20080000` from Binary Ninja's **Python console** (`Plugins -> Python Console`) — no command port needed:

   ```
   dbg.write_memory(0x20080000, b"foo: %d\r\n\x00")
   ```

   `dbg.write_memory(address, bytes)` is Binary Ninja's debugger memory-write API; it returns `True` on success. That writes the bytes `66 6f 6f 3a 20 25 64 0d 0a 00` = `"foo: %d\r\n\0"` .
3. Point `r0` at that string:

   ```
   dbg.set_reg_value("r0", 0x20080000)
   ```

   (Or right-click `r0` in the **Registers** widget, press `E` , type `0x20080000` , and press Enter.)
4. Move the breakpoint past the call in the GUI (remove it at `0x1000023e` , set one at `0x10000242`) and click **Resume**. The core runs `printf` with `r0` pointing at your RAM string and `r1 = 0x2b` , so this iteration prints:

   ```
   foo: 43
   ```

   then stops at `0x10000242` .

Like the value hack, this is **one iteration only**: the loop reloads `r0` (and `r1`) from flash on every pass, so the next line is `age: 43` again. The permanent version is the static patch in Step 19b.

Step 15: Why the hack reverts (and why we patch next)

Press **Resume**. The loop branches back to `0x1000023a` , which reloads `movs r1, #43` , so the next line is `age: 43` . The live edit changed one iteration only. There is no variable in memory to change; the value is baked into the instruction. To make `age: 70` permanent we must patch the instruction. That is the static pass.

Press **Pause** to stop the output flood.

Step 15b: Kill the debugger and OpenOCD

The live hack is done. Do this **before** the static pass.

1. In the **Debugger** sidebar, click the **X** (**Kill**) (or `Debugger -> Kill`) to disconnect Binary Ninja.
2. **Kill does not stop the OpenOCD process** — `debug-server.sh` started it separately, and it keeps running and holding the probe. Stop it from the Binary Ninja console:

   macOS / Linux:

   ```
   import subprocess
   subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
   ```

   Windows:

   ```
   import subprocess
   subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the
   probe
   ```

3. Confirm nothing is left: `pgrep -fl openocd` (macOS/Linux) prints nothing.

From a terminal it is the same: `pkill -TERM -f openocd`, or `Get-Process openocd | Stop-Process` on Windows.

Part 4: Static — Resolve the Functions in Binary Ninja and Patch (Project 1)

Step 16: Resolve the functions in the Binary Ninja GUI

We now name the functions in Binary Ninja using the ELF symbol map from Step 4. Binary Ninja loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	Binary Ninja action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in Binary Ninja (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_10002f54()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

The decompiler header then shows the real prototype, and calls to the function read cleanly instead of `sub_<addr>()`. `N` is only for renaming without touching the type; `Y` alone sets both the name and the type.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. Binary Ninja parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The view jumps there; the cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter. That sets the name to `main` and the type to `int(void)`.

> **Binary Ninja shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and Binary Ninja's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `stdio_init_all`

1. `G` -> `0x10002f54`.
2. `Y` -> `bool stdio_init_all(void)`.

It returns `bool`, not `void` — the ELF says `_Bool stdio_init_all(void)`. Our `main` ignores the return value, so the decompiler still reads cleanly.

Worked example: `uart_init`

1. `G` -> `0x10000e10`.
2. `Y` -> `uint uart_init(uart_inst_t *uart, uint baudrate)`.

Worked example: `__wrap_printf`

1. `G` -> `0x100030e4`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

> `printf` **in our source is** `__wrap_printf` **in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper.

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK. `main` only calls `stdio_init_all` and `printf`, so we follow that chain down: `stdio_init_all` pulls in the stdio/UART setup, and `printf` lands in the SDK's `__wrap_printf`.

The call chain for this project:

```
main
├── stdio_init_all
│   ├── stdio_uart_init ── gpio_set_function, uart_init
│   ├── stdio_set_driver_enabled
│   ├── stdio_out_chars_crlf
│   ├── stdio_put_string ── strlen
│   └── time_us_64
└── __wrap_printf ── __wrap_vprintf
```

Project 1 — resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`
`0x10002cfc`	`exit`	`void exit(int)`
`0x10002d04`	`runtime_init`	`void runtime_init(void)`
`0x10002f54`	`stdio_init_all`	`bool stdio_init_all(void)`
`0x100032a0`	`stdio_uart_init`	`void stdio_uart_init(void)`
`0x10002f2c`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`
`0x10002d30`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`
`0x10002e40`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`
`0x100030e4`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`
`0x10003020`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`
`0x10000248`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`
`0x10000e10`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`
`0x10000da0`	`time_us_64`	`uint64_t time_us_64(void)`
`0x100033e0`	`strlen`	`size_t strlen(const char*)`

A `void` return type may not stick — here is the fix. Binary Ninja treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type Binary Ninja thinks it can infer.

Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:

```
from binaryninja import ReturnValue, Type
fn = bv.get_function_at(0x1000015c)
if fn is not None:
    fn.return_value = ReturnValue(Type.void())
```

That holds `_reset_handler` at `void` even after reanalysis (verified live). Setting the signature and the return type both failing while `return_value` succeeds looks like a bug or inconsistency in this build (BN 6.0.10601). If it still will not stick, leave it — it does not affect the rest of the lesson.

`__wrap_printf` is the real symbol. `printf` in our source compiles to the SDK's `__wrap_printf` (which forwards to `__wrap_vprintf`). Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

`stdio_init_all` returns `bool`, not `void` — `_Bool stdio_init_all(void)` in the ELF. The `main` source ignores the return value, so the decompiler still reads fine.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into Binary Ninja's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature);  None means "leave the type alone"
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x10002cfc: ("exit",                      "void exit(int)"),
    0x10002d04: ("runtime_init",              "void runtime_init(void)"),
    0x10002f54: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x100032a0: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10002f2c: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10002d30: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10002e40: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x100030e4: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x10003020: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x10000248: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x10000e10: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x10000da0: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x100033e0: ("strlen",                    "size_t strlen(const char*)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    if sig:
        f = bv.get_function_at(addr)
        if f is not None:
            f.set_user_type(sig)
```

SDK type names (`stdio_driver_t` , `gpio_function_t` , `uart_inst_t` , plus `uint` and `va_list`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first (an opaque `struct` / `enum` / `typedef` is enough to parse). If you add a function that uses another SDK type, add a definition for it to that block too.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main`. It reads:

```
int32_t main(void)
{
    stdio_init_all();
    do
    {
        printf("age: %d\r\n", 0x2b);
    } while (true);
}
```

The `0x2b` is the value we edited live. Now make it permanent.

Step 18: Patch `0x2b` to `0x46` in the GUI

Go to `0x1000023a`:

```
1000023a: 212b   movs  r1, #43 ; 0x2b
```

The halfword is `0x212b`, stored little-endian as `2b 21`. The immediate is the low byte, so the byte at the instruction's own address is `0x2b`. Change it to `0x46` (70).

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x1000023a` and change the byte `2B` to `46`.
4. Return to the linear view, right-click the function -> `Reanalyze`.

Option B — Python console:

```
bv.write(0x1000023a, b"\x46")
print(hex(bv.read(0x1000023a, 1)[0]))  # -> 0x46
```

After reanalysis the instruction reads `movs r1, #70`.

Step 18b: Patch the string `age:` to `foo:` in the GUI

The format string `"age: %d\r\n"` starts at `0x100034a0`. Its first three bytes are `61 67 65` (`age`). Change them to `66 6f 6f` (`foo`), leaving the `: %d\r\n` tail untouched, so the line prints `foo: 70`.

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x100034a0` and change the three bytes `61 67 65` to `66 6f 6f`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x100034a0, b"foo")
print(bv.read(0x100034a0, 10))  # -> b'foo: %d\r\n\x00'
```

Keep the replacement exactly three bytes. If you use a shorter string you must pad it, or `%d` shifts and `printf` reads the wrong argument. A longer string would overwrite the `: %d` tail.

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x0005_intro-to-variables", "build"), "0x0005_intro-to-
variables-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 15292 /.../build/0x0005_intro-to-variables-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base Binary Ninja loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x3bbc` = 15292). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.
- Reading `seg.start` for `seg.data_length` bytes therefore grabs exactly the image.

Two gotchas this avoids:

- **No relative path.** Binary Ninja's Python console runs with a read-only working directory (inside the app bundle), so `open("0x0005_intro-to-variables-h.bin", "wb")` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/` — no machine-specific path and no database needed.
- **Read the image, not the whole view.** `bv.read(bv.start, bv.length)` spans the entire mapped range (`0x10000000`..`0xe008000c`), which is not the image. The segment's `data_length` is the image size.

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0005_intro-to-variables-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0005_intro-to-variables-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the Binary Ninja console** — it is a normal Python interpreter, so you never have to leave the app. `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x0005_intro-to-variables", "build"))   # the project build dir
> (writable)
> sys.argv = ["uf2conv.py", "0x0005_intro-to-variables-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")   # path to your uf2conv.py
> ```
>
> This writes `hacked.uf2` next to the `.bin`, ready to drag onto the Pico.

Step 21: Flash and verify `age: 70`

Hold **BOOTSEL**, plug in the Pico 2, and drag `hacked.uf2` onto the `RP2350` drive. Open the serial monitor:

```
age: 70
age: 70
age: 70
...
```

43 became 70, permanently, with one byte changed and no source code.

Faster: flash over the Debug Probe (no BOOTSEL). The repo's `flash.sh` writes the raw `.bin` straight into XIP flash over SWD (`program <bin> 0x10000000 verify reset exit`), so you never touch BOOTSEL or a UF2. Run it from a terminal (`./flash.sh <bin>`), or from the Binary Ninja console **without freezing it** — use `subprocess.Popen`, which returns immediately, and send OpenOCD's output to a log file. (`subprocess.run` blocks the console until the flash finishes; do not use it here.)

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
bin_path = os.path.join(os.path.join(root, "0x0005_intro-to-variables", "build"), "0x0005_intro-
to-variables-h.bin")
log = os.path.join(os.path.join(root, "0x0005_intro-to-variables", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

The `pkill` frees the probe first; on Windows use `subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])`.

The console is free the moment this returns. Check it with `print(p.poll())` (`None` = still running, `0` = done) or read `flash.log` — success ends with `** Verified OK **`. (Verified: `Popen` returns in ~1 ms; the flash itself takes ~2 s.)

The same non-blocking form without the script:

```
import os, subprocess
ocd = os.path.expanduser("~/.pico-sdk/openocd/0.12.0+dev")
bin_path = os.path.join(os.path.join(root, "0x0005_intro-to-variables", "build"), "0x0005_intro-
to-variables-h.bin")
log = os.path.join(os.path.join(root, "0x0005_intro-to-variables", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([f"{ocd}/openocd", "-s", f"{ocd}/scripts",
    "-f", "interface/cmsis-dap.cfg", "-f", "target/rp2350.cfg",
    "-c", "adapter speed 5000",
    "-c", f"program {bin_path} 0x10000000 verify reset exit"],
    stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

The Debug Probe is single-owner. If Binary Ninja is still attached (the `debug-server.sh` OpenOCD is running), the flash cannot grab the probe. Detach in Binary Ninja and stop that OpenOCD first:

```
# macOS / Linux
pkill -TERM -f openocd
```

```
# Windows
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Success looks like `Programming Finished` -> `Verified OK` -> `Resetting Target`. On Windows use `flash.ps1` (`.\flash.ps1 -Bin <path>`) the same way.

Part 5: Dynamic — Break at `main` and Hack Live (Project 2)

Step 22: Reflash Project 2 and reload Binary Ninja

Part 4 left the Pico running the patched Project 1 image. Put the original Project 2 back and start a fresh session.

1. Stop any running debug server so the flash script can use the probe:

```
# macOS / Linux
pkill -TERM -f openocd
```

```
# Windows
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

2. Flash the original Project 2 image:

```
# macOS / Linux
./flash.sh 0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin
```

```
# Windows
.\flash.ps1 -Bin 0x0008_uninitialized-variables\build\0x0008_uninitialized-variables.bin
```

Or do steps 1–2 from the Binary Ninja console (the active view is still Project 1, so take the repo root from it and point at the Project 2 `.bin`):

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step
3)
bin_path = os.path.join(root, "0x0008_uninitialized-variables", "build",
"0x0008_uninitialized-variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing Project 2 in the background; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step
3)
bin_path = os.path.join(root, "0x0008_uninitialized-variables", "build",
"0x0008_uninitialized-variables.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing Project 2 in the background; log:", log)
```

3. Start the debug server again (Step 10) and wait for `Listening on port 3333`.

4. Load Project 2 and save its database — see Step 22b.
5. Connect Binary Ninja again (Step 11): adapter **GDB MI**, IP `127.0.0.1`, port `3333`.

Confirm the Pico prints `age: 0` and blinks the red LED.

Step 22b: Load Project 2 into Binary Ninja and save the database

Exactly like Steps 7–8, but for Project 2. **Use** `File -> Open with Options...` (not plain `File -> Open`), select `0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin`, and set:

- **Architecture:** `thumb2`
- **Platform:** `thumb2`
- **Base Address:** `0x10000000`

Click **Open**. Then press `G`, type `0x10000000`, and confirm the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you see data at `0x00000000`, close the tab and redo it with `Open with Options`.

Save it with `File -> Save As...` as `0x0008_uninitialized-variables.bndb` (next to the `.bin`). From now on open the `.bndb`, not the `.bin`; save with `Cmd+S` / `Ctrl+S` after every rename or patch.

> **Console equivalent:**
>
> ```
> load("0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin",
>      options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Then resolve the functions for Project 2 the same way as Project 1 — Step 26.

Step 23: Break at `main`

`main` is at `0x10000234` in this project too. The GUI sets breakpoints fine (Step 13); the only caution is not to drive `reset run` from the port while Binary Ninja is attached (it desyncs Binary Ninja's view). Use `BP_ADDR`, which arms `main` before Binary Ninja connects:

1. Stop the server (Ctrl-C), then start it parked at `main`:

```
# macOS / Linux
BP_ADDR=0x10000234 ./debug-server.sh
```

```
# Windows
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Or restart it from the Binary Ninja console — kill any running server, then start it parked at `main`:

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
log = os.path.join(root, "openocd.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # kill any running server first
subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                 env=dict(os.environ, BP_ADDR="0x10000234"),
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("OpenOCD restarted parked at main; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
log = os.path.join(root, "openocd.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # kill any running server first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "debug-server.ps1")], cwd=root,
                 env=dict(os.environ, BP_ADDR="0x10000234"),
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("OpenOCD restarted parked at main; log:", log)
```

2. Connect Binary Ninja (Step 11): adapter **GDB MI**, IP `127.0.0.1`, port `3333`.

The target is already halted at `main` when Binary Ninja connects, and the sidebar reads `Stopped at 0x10000234`.

> If you instead want to reach `main` on an already-connected session, you must Detach, send `reset run` from the port, then reconnect. Arming `main` and resetting while attached leaves the sidebar showing a stale address.

`main` sets up GPIO 16 and then loops: print `age`, turn the LED on, sleep, turn it off, sleep. The whole loop is one function because `blink_and_print` was inlined:

```
10000234 <main>:
10000234: b538         push  {r3, r4, r5, lr}
10000236: f002 ff49    bl    0x100030cc        ; stdio_init_all
1000023a: 2010         movs  r0, #16           ; LED_PIN
1000023c: f000 f83a    bl    0x100002b4        ; gpio_init
10000240: f04f 0501    mov.w r5, #1
10000244: 2310         movs  r3, #16           ; LED_PIN
10000246: ec45 3044    mcrr  0, 4, r3, r5, cr4 ; gpio_set_dir(16, OUT)
1000024a: 2100         movs  r1, #0            ; age
1000024c: 4809         ldr   r0, [pc, #36]     ; @ 0x10000274 -> "age: %d\r\n"
1000024e: f003 f805    bl    0x1000325c        ; __wrap_printf
10000252: 2410         movs  r4, #16           ; LED_PIN
10000254: ec45 4040    mcrr  0, 4, r4, r5, cr0 ; gpio_put(16, 1)
10000258: f44f 70fa    mov.w r0, #500
1000025c: f000 fd58    bl    0x10000d10        ; sleep_ms
10000260: f04f 0300    mov.w r3, #0
10000264: ec43 4040    mcrr  0, 4, r4, r3, cr0 ; gpio_put(16, 0)
10000268: f44f 70fa    mov.w r0, #500
1000026c: f000 fd50    bl    0x10000d10        ; sleep_ms
10000270: e7eb         b.n   0x1000024a
10000272: bf00         nop
10000274: 10003618     .word 0x10003618
```

Look at the **Registers** widget at `0x1000024e`: `r1` is `0`, which is why the Pico prints `age: 0`.

Step 24: Inspect the GPIO registers live

The GPIO hardware is memory-mapped. Go to each address and watch it change as you step:

Address	Block	Role
`0x40028000`	`IO_BANK0`	pin function select and status
`0x40038000`	`PADS_BANK0`	pad configuration
`0xd0000000`	`SIO`	single-cycle GPIO block driven by `mcrr`

Step Over through `0x10000254` (`mcrr 0, 4, r4, r5, cr0`) and watch the SIO output register change: this is `gpio_put(16, 1)` turning the red LED on at the hardware level.

Step 25: HACK IT LIVE — change the printed value

1. Press `G`, go to `0x1000024e` (the `bl __wrap_printf`).
2. Set a **hardware execute** breakpoint at `0x1000024e` in the GUI (`Debugger -> Add Hardware Breakpoint...`; not `F2`). Note `0x1000024e` — Project 2's loop sits at a different address than Project 1's.
3. Click **Resume** in Binary Ninja. The target is already looping, so the breakpoint fires on the next pass. Binary Ninja stops with `r1 = 0`.
4. Set `r1` to `0x42` (66):

   ```
   dbg.set_reg_value("r1", 0x42)
   ```

   (Or right-click `r1` in the **Registers** widget, press `E`, type `42`, and press Enter.)
5. **Move the breakpoint past the call.** `0x10000252` is the instruction right after the `bl __wrap_printf`. Remove the breakpoint at `0x1000024e` and set a hardware breakpoint at `0x10000252`, then click **Resume**. The core runs `printf` with `r1 = 0x42` and stops at `0x10000252`. (Not **Step Over** — it steps into the call on this symbol-less `.bin`, and a breakpoint left on the current PC re-traps the step; Step 13 explains both.)
6. Look at your serial monitor and the **Target** tab:

   ```
   age: 66
   ```

Press **Resume** and the next iteration prints `age: 0` again, because the loop reloads `movs r1, #0` each pass. The live hack is temporary; the static patch makes it permanent.

Step 25b: HACK THE STRING LIVE — change `age:` to `foo:`

Same idea as Project 1, different addresses. Here the format string is at `0x10003618` and the `printf` call is at `0x1000024e`.

1. Hit the breakpoint at `0x1000024e` as in Step 25. At the stop, `r0 = 0x10003618` and `r1 = 0`.
2. Write the replacement string to free RAM at `0x20080000` from the **Python console** (`dbg.write_memory` — no command port needed):

   ```
   dbg.write_memory(0x20080000, b"foo: %d\r\n\x00")
   ```

   Bytes `66 6f 6f 3a 20 25 64 0d 0a 00` = `"foo: %d\r\n\0"`.
3. Point `r0` at that string:

   ```
   dbg.set_reg_value("r0", 0x20080000)
   ```

   (Or right-click `r0` in the **Registers** widget, press `E`, type `0x20080000`, and press Enter.)
4. Move the breakpoint past the call in the GUI (remove it at `0x1000024e`, set one at `0x10000252`) and click **Resume**. This iteration prints:

   ```
   foo: 0
   ```

   then stops at `0x10000252`. One iteration only — the loop reloads `r0` each pass. The permanent version is the static patch in Step 28b.

Step 25c: Kill the debugger and OpenOCD

Same as Step 15b: click the **X** (**Kill**) in the **Debugger** sidebar (or `Debugger -> Kill`), then stop OpenOCD from the Binary Ninja console:

macOS / Linux:

```
import subprocess
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
```

Windows:

```
import subprocess
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the probe
```

Part 6: Static — Resolve the Functions and Patch (Project 2)

Step 26: Resolve the functions in the Binary Ninja GUI

Same two keys as Step 16 — `G` to the address, then `Y` (Change Type) to set the prototype — using the Project 2 ELF symbol map from Step 4.

The mechanics are identical to Step 16, so here are the worked examples for the functions that are specific to this project.

`main`

1. `G` -> `0x10000234`.
2. `Y` -> `int main(void)` (Binary Ninja shows `int32_t main(void)` — the same 32-bit `int`).

`gpio_init`

1. `G` -> `0x100002b4`.
2. `Y` -> `void gpio_init(uint gpio)`.

`sleep_ms`

1. `G` -> `0x10000d10`.
2. `Y` -> `void sleep_ms(uint32_t ms)`.

`stdio_init_all` and `__wrap_printf`

Same as Project 1, different addresses: `stdio_init_all` at `0x100030cc` (`bool stdio_init_all(void)`), and `__wrap_printf` at `0x1000325c` (`int __wrap_printf(const char *fmt, ...)`).

Then work down the table the same way.

Same idea as Project 1: **our code plus what it calls**, not the whole SDK. The call chain here is one function longer because `main` also drives the GPIO and sleeps:

```
main
├── stdio_init_all
│   ├── stdio_uart_init ── gpio_set_function, uart_init
│   ├── stdio_set_driver_enabled
│   ├── stdio_out_chars_crlf
│   ├── stdio_put_string ── strlen
│   └── time_us_64
├── gpio_init
├── __wrap_printf ── __wrap_vprintf
└── sleep_ms
```

Two things in this project have **no symbol of their own**, because the compiler inlined them into `main`:

- `blink_and_print` — the `static` helper in our own source is inlined, so there is no `blink_and_print` address to rename. You see its body directly inside `main`.
- `gpio_set_dir` and `gpio_put` — these are `static inline` in the SDK headers, so they compile to the `mcrr`/SIO writes you see in `main` rather than to calls.

Project 2 — resolve every function in that chain:

Address	Rename to (N)	Signature (Y)
0x1000015c	_reset_handler	void _reset_handler(void)
0x10000186	platform_entry	void platform_entry(void)
0x1000019a	data_cpy	void data_cpy(void*, void*, void*)
0x100001e4	_init	void _init(void)
0x10000210	frame_dummy	void frame_dummy(void)
0x10000234	main	int main(void)
0x100002b4	gpio_init	void gpio_init(uint)
0x10000278	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x10000d10	sleep_ms	void sleep_ms(uint32_t)
0x10002e74	exit	void exit(int)
0x10002e7c	runtime_init	void runtime_init(void)
0x100030cc	stdio_init_all	bool stdio_init_all(void)
0x10003418	stdio_uart_init	void stdio_uart_init(void)
0x100030a4	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10002ea8	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10002fb8	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x1000325c	__wrap_printf	int __wrap_printf(const char*, ...)
0x10003198	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x10000f88	uart_init	uint uart_init(uart_inst_t*, uint)

Address	Rename to (N)	Signature (Y)
`0x10000ef4`	`time_us_64`	`uint64_t time_us_64(void)`
`0x10003558`	`strlen`	`size_t strlen(const char*)`

Python console shortcut (resolves name **and** type):

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature);  None means "leave the type alone"
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x100002b4: ("gpio_init",                 "void gpio_init(uint)"),
    0x10000278: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x10000d10: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x10002e74: ("exit",                      "void exit(int)"),
    0x10002e7c: ("runtime_init",              "void runtime_init(void)"),
    0x100030cc: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x10003418: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x100030a4: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10002ea8: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10002fb8: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x1000325c: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x10003198: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x10000f88: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x10000ef4: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x10003558: ("strlen",                    "size_t strlen(const char*)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    if sig:
        f = bv.get_function_at(addr)
        if f is not None:
            f.set_user_type(sig)
```

The decompiler now shows `main` initializing GPIO 16 and looping. We make two changes:

- **Move the LED from GPIO 16 to GPIO 17** by patching three `0x10` immediates.
- **Change the printed value from 0 to 66** by patching one `0x00` immediate.

Step 27: Patch 1 — move the LED from GPIO 16 to GPIO 17

GPIO 16 is the red LED; GPIO 17 is the green LED. The pin number appears in three instructions. Change the low byte of each from `10` to `11`:

Address	Instruction	Bytes before	Bytes after	Role
`0x1000023a`	`movs r0, #16`	`10 20`	`11 20`	pin passed to `gpio_init`
`0x10000244`	`movs r3, #16`	`10 23`	`11 23`	pin used by `gpio_set_dir`
`0x10000252`	`movs r4, #16`	`10 24`	`11 24`	pin used by `gpio_put` in the blink loop

In the **Hex** view (lock off), change each byte and reanalyze. Or in the Python console:

```
for addr in (0x1000023a, 0x10000244, 0x10000252):
    bv.write(addr, b"\x11")
```

> **All three are required.** If you patch only the `gpio_set_dir` site, pin 17's output driver is enabled but `gpio_put` still drives pin 16, whose driver was never enabled. Nothing lights up. This is the most common mistake in this lesson.

Step 28: Patch 2 — change the printed value from 0 to 66

`main` loads `age = 0` with `movs r1, #0` at `0x1000024a`. Change the immediate byte from `00` to `42` (`0x42` = 66):

```
bv.write(0x1000024a, b"\x42")
```

Verify all four patches:

```
for addr in (0x1000023a, 0x10000244, 0x10000252, 0x1000024a):
    print(hex(addr), hex(bv.read(addr, 1)[0]))
# -> 0x1000023a 0x11
# -> 0x10000244 0x11
# -> 0x10000252 0x11
# -> 0x1000024a 0x42
```

Step 28b: Patch the string `age:` to `foo:`

The format string starts at `0x10003618`; change its first three bytes `61 67 65` (`age`) to `66 6f 6f` (`foo`):

```
bv.write(0x10003618, b"foo")
print(bv.read(0x10003618, 10))  # -> b'foo: %d\r\n\x00'
```

Exactly three bytes, same rule as Project 1: a shorter string must be padded, a longer one overwrites the `: %d` tail.

Step 29: Export, convert, and flash

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size from the view itself
out = os.path.join(os.path.join(root, "0x0008_uninitialized-variables", "build"),
"0x0008_uninitialized-variables-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 15668 /.../build/0x0008_uninitialized-variables-h.bin
```

Same as Project 1: `seg.start` is the load base and `seg.data_length` is the image size (here `0x3d34` = 15668) — both read from the view, and no relative path (the console's CWD is read-only).

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0008_uninitialized-variables-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0008_uninitialized-variables-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Or run the conversion from the Binary Ninja console, exactly as in Step 20 (`os.chdir` to the build dir, then `runpy.run_path("../../uf2conv.py", run_name="__main__")` with `sys.argv` set to the arguments above).

Hold **BOOTSEL**, plug in the Pico 2, drag `hacked.uf2` onto the `RP2350` drive. Or flash the `.bin` over the Debug Probe with SWD — no BOOTSEL — from the console, exactly as in Step 21 (stop any running OpenOCD first, and use `Popen`, not `run`, so the console is not blocked):

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
bin_path = os.path.join(os.path.join(root, "0x0008_uninitialized-variables", "build"),
"0x0008_uninitialized-variables-h.bin")
log = os.path.join(os.path.join(root, "0x0008_uninitialized-variables", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Step 30: Verify

Open the serial monitor:

```
age: 66
age: 66
age: 66
...
```

The **green LED on GPIO 17** now blinks instead of the red one.

We changed the printed value and moved the LED, with four bytes and no source code.

Cheatsheet

Binary Ninja GUI actions

Action	How
Go to address	`G`
Rename function/symbol	`N`
Set type or signature	`Y`
Add comment	`;`
Open Hex view	`View -> Hex`
Enable hex editing	Toggle the lock in the status bar
Reanalyze after a patch	Right-click function -> `Reanalyze`
Edit a register live	`dbg.set_reg_value("r1", 0x46)` in the Python console (or right-click the register, press `E`, type hex, Enter)
Set a breakpoint	`Debugger -> Add Hardware Breakpoint...` (hardware execute). Do **not** use `F2` — software breakpoints cannot be written to read-only flash.
Move a breakpoint	Remove it and set it at the new address in the GUI (command-port fallback: `rbp <old addr>` then `bp <new addr> 2 hw`)
Confirm what is armed	The **Breakpoints** widget lists it (command-port fallback: `mdw 0xE0002000 8`, each armed breakpoint shows as `<addr | 1>`)
Apply the ELF symbol map	Paste the Python snippet from Step 16 / 26 into the Python Console

OpenOCD server and reset

The server runs with `gdb_breakpoint_override hard` so that flash-writes are never attempted. Breakpoints in this lab are set in the Binary Ninja GUI through the **GDB MI** adapter (Step 13). The command-port rows below are the fallback if you use the **GDB RSP** adapter instead.

Action	Command
Connect to the OpenOCD prompt (fallback)	`nc 127.0.0.1 4444` (or `telnet 127.0.0.1 4444`)
Reset and run (command port)	`reset run`
Check core state (command port)	`targets`
Set a breakpoint in the GUI	`Debugger -> Add Hardware Breakpoint...` (hardware execute; `F2` software breakpoints do not work on flash)

Action	Command
(fallback) Add a breakpoint without the GUI	`bp <addr> 2 hw`
Remove one breakpoint	`rbp <addr>` — **address only, no length, no** `hw`
Remove every breakpoint	`rbp all`
Start the server parked at `main`	macOS/Linux: `BP_ADDR=0x10000234 ./debug-server.sh` — Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1` (one-shot)
Start the server parked in the loop	macOS/Linux: `BP_ADDR=0x1000023e ./debug-server.sh` — Windows: `$env:BP_ADDR="0x1000023e"; .\debug-server.ps1` (`0x1000024e` for Project 2)
Break on the loop in a running target	set a hardware breakpoint in the GUI at the loop address, then **Resume** — repeatable
Make Binary Ninja stepping work	`rp2350.dap.core0 configure -rtos none` (already in the scripts)
Step without re-trapping	move the breakpoint off the current PC first, then **Step Into**/**Step Over**
Reset without desyncing Binary Ninja	**Detach**, `reset run` on the port, reconnect — never `reset run` while attached

Every address and byte we changed

Project	Address	Before	After	Effect
`0x0005`	`0x1000023a`	`2b`	`46`	prints `age: 70`
`0x0008`	`0x1000023a`	`10`	`11`	`gpio_init` configures GPIO 17
`0x0008`	`0x10000244`	`10`	`11`	`gpio_set_dir` enables GPIO 17
`0x0008`	`0x10000252`	`10`	`11`	`gpio_put` drives GPIO 17
`0x0008`	`0x1000024a`	`00`	`42`	prints `age: 66`
`0x0005`	`0x100034a0`	`61 67 65`	`66 6f 6f`	string prints `foo:` instead of `age:`
`0x0008`	`0x10003618`	`61 67 65`	`66 6f 6f`	string prints `foo:` instead of `age:`

Raw image facts

Item	Value
Build type	`Release`
Load base address	`0x10000000`
Project 1 size	`15292` bytes
Project 2 size	`15668` bytes
Fixed `main` anchor (both projects)	`0x1000018c` (reset handler middle `blx`)

Item	Value
`main` (both projects)	`0x10000234`
`printf` call, Project 1	`0x1000023e`
`printf` call, Project 2	`0x1000024e`
RP2350 UF2 family ID	`0xe48bff59`

Troubleshooting

Binary Ninja hangs or crashes when you connect (macOS 27)

Three different causes have been seen on this setup; check them in this order.

- **A breakpoint set before connecting.** With the **GDB MI** adapter, if the binary view already has a breakpoint, the session hangs. Start parked with `BP_ADDR`, connect, then add breakpoints (see the next entry).
- **The wrong GDB executable.** Point **Full GDB Executable Path** at the **14.2.rel1** toolchain (Step 11). The 13.3.rel1 build (what `/opt/homebrew/bin/arm-none-eabi-gdb` symlinks to) did not connect in testing.
- **The LLDB adapter.** A crash report with `libdebuggercore.dylib -> std::terminate() -> abort()` and `liblldb` in the stack is the **LLDB** adapter, not GDB MI. Avoid LLDB on this setup.

Use GDB MI, with the 14.2.rel1 path above. If it still fails, fall back to plain `arm-none-eabi-gdb` against the same server — the addresses and register values are identical to the GUI steps.

If Binary Ninja hangs, force-quit it; the connect dialog has no working Cancel. The static steps (resolve, patch, export, flash) never touch the debugger and always work.

The GUI refuses to set a breakpoint (GDB RSP adapter only)

If you are on the **GDB RSP** adapter, the GUI cannot set breakpoints on this target. That adapter is Binary Ninja's own minimal RSP client and sends a **1-byte** breakpoint (`Z0,<addr>,1`); the Cortex-M33 comparators need 2 bytes, so OpenOCD answers `only breakpoints of two bytes length supported`. It affects every address, both `Toggle Breakpoint` and `Add Hardware Breakpoint`, and the dialog's **Size** field is disabled. `gdb_breakpoint_override` makes no difference.

Fix: use the GDB MI adapter (Step 11). It drives real GDB, which sends the correct length, so GUI breakpoints just work. If you must stay on GDB RSP, arm breakpoints from the command port after connecting (`bp <addr> 2 hw`) — but the lab uses GDB MI and does not need that.

GDB MI hangs when you connect (a breakpoint already existed)

With the **GDB MI** adapter, if Binary Ninja already has a breakpoint set when you connect, the session **hangs**. This is a Binary Ninja bug. The working order is:

1. Start the server parked, e.g. `BP_ADDR=0x10000234 ./debug-server.sh` (Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1`).
2. Connect with the **GDB MI** adapter.
3. Only *then* set hardware breakpoints in the UI.

Never have a breakpoint in the binary view before the GDB MI connection. If it hangs, quit Binary Ninja, restart the server with `BP_ADDR`, and connect again before adding any breakpoints.

Step Into / Step Over does nothing (PC never moves)

Two causes have been seen on this target.

1. **A breakpoint on the current PC re-traps the step.** OpenOCD's step-over-breakpoint logic fails with `Duplicate Breakpoint address` and the PC stays put. Fix: move the breakpoint off the current PC (in the GUI), then step.
2. **The `hwthread` RTOS (GDB RSP adapter only).** With the **GDB RSP** adapter, OpenOCD can log `fake step thread 0` and reply without stepping, because the RP2350 config's `-rtos hwthread` makes the current thread id 1 while Binary Ninja sends thread id 0. Fix: `rp2350.dap.core0 configure -rtos none` (the launcher scripts already pass this). **GDB MI does not hit this.**

To tell them apart, turn on OpenOCD logging (`log_output /tmp/ocd.log`, then `debug_level 3` on the command port) and look for `fake step` versus `Duplicate Breakpoint`.

`zsh: bad CPU type in executable: cmake`

An Intel `x86_64` tool is on your `PATH` on Apple Silicon. Run Step 2: `export PATH="/opt/homebrew/bin:$PATH"`, then `hash -r`. Add it to `~/.zshrc` to make it permanent.

My addresses do not match this guide

You probably built `Debug`. This lesson is a `Release` build. Re-run Step 3 with `-DCMAKE_BUILD_TYPE=Release`. A `Debug` build moves the SDK functions and keeps `blink_and_print` separate, so Project 2's `main` is not at `0x10000234`.

A breakpoint never fires

First, confirm you actually set one, and that it is a **hardware** breakpoint. With the **GDB MI** adapter, `Debugger -> Add Hardware Breakpoint...` (hardware execute) should land in the **Breakpoints** widget. If nothing lands, or the core keeps running, you probably used `F2` (`Toggle Breakpoint`) — that is a software breakpoint and cannot be written to read-only flash, so it never installs. Also check you are on **GDB MI**, not **GDB RSP** (the GDB RSP adapter cannot set breakpoints on this target at all).

Then check the order and the state:

- **Arm it only after Binary Ninja is connected.** OpenOCD flushes every breakpoint when a client attaches (`breakpoint_remove_all_internal -> Delete all breakpoints`), so anything armed earlier is gone. This also applies to `BP_ADDR` on the startup command line, and to `hbreak` followed by `detach` in GDB.
- **Verify it is armed:** `mdw 0xE0002000 8`. You should see your address with the low bit set (`0x10000234` -> `0x10000235`). All zeros means nothing is armed — re-read this first, because it distinguishes "not armed" from "armed but never reached".
- **Is the core running?** `poll` on the command port should not report a halt. If it is stopped, click **Resume**.
- **Does the address get reached again?** `main` runs once per reset, so use `BP_ADDR` at startup (Step 10) rather than `reset run` while attached. Loop addresses such as `0x1000023e` fire on the next pass with no reset — arm them and click **Resume** in Binary Ninja.
- **With GDB MI the stop is reported as** `Breakpoint` and appears in the **Breakpoints** widget, because GDB really did set it. (On the old **GDB RSP** workaround the stop showed as `SingleStep` with an empty

widget, because the breakpoint was armed behind Binary Ninja's back.)

I edit `r1` (or another register) and it reverts

`main` reloads the value at the top of every loop iteration — `movs r1, #43` at `0x1000023a` runs right before the `printf` at `0x1000023e`. So `r1` is only `0x46` for the instant between your edit and the next pass; then it is `0x2b` again. The edit sticks only if the core is **genuinely stopped** at the breakpoint and stays stopped.

If it keeps reverting, the core is running, which almost always means the breakpoint is not installed — usually because it is a **software** breakpoint (`F2`) that cannot be written to read-only flash. Use `Debugger -> Add Hardware Breakpoint...` (hardware execute). Confirm with `mdw 0xE0002000 4` on the command port: a hardware breakpoint shows as `0x1000023f` ; all zeros means nothing is armed.

> **The Registers widget is a snapshot, not a live view.** Binary Ninja reads the registers at each stop and shows that snapshot; it does not poll the target, and there is no "refresh registers" command (only "Force Update Memory Cache", which is for memory). So a value changed outside Binary Ninja will not appear until the next stop.

The serial capture is garbage on macOS

Reading `/dev/cu.usbmodem*` with a bare `read()` returns garbage. Set **raw termios at 115200** first: clear canonical/echo flags, set `CLOCAL|CREAD` , and `B115200` on input and output. `screen /dev/cu.usbmodem* 115200` does all of this for you; a script must call `tcsetattr` itself. Once set, the capture reads clean `age: 43` lines.

It worked for a second, then stopped (Binary Ninja's view desyncs)

This is the most common failure, and it has one main cause: **driving the core from the OpenOCD command port while Binary Ninja is connected.**

- If you send `reset run` from the port while attached, the core resets, runs, and halts at your breakpoint — but Binary Ninja never receives the stop event. Its sidebar keeps showing the *previous* location, so **Step** and **Resume** act on a stale PC and appear to do nothing. Verified: target at `0x1000023e` while the sidebar still read `Stopped at 0x10003020` .
- If the OpenOCD process dies (or you restart it) while attached, Binary Ninja keeps believing it is connected: the sidebar stays, but the menu shows **Pause** enabled and **Resume**/**Step** disabled because Binary Ninja last saw the target *running*.

Recovery: **Detach, then reconnect.** If Detach does nothing (the connection is already dead), restart Binary Ninja — its menu still shows a session that no longer exists.

Prevention:

- Stop at `main` with `BP_ADDR` on a fresh server start, not with `reset run` while attached.
- For loop addresses, set the breakpoint in the GUI and click **Resume**. Let Binary Ninja be the thing that starts the core.
- If you must reset, **Detach first**, `reset run` , then reconnect.
- Never leave a breakpoint on the PC you are about to step or resume from (see the stepping section above).

> **If the stop is at `0x1000320c` rather than your breakpoint,** you stopped inside `stdio_uart_out_flush` , not at `main` . See the next section.

The target "blows past" `main` and stops at `0x1000320c` instead

`0x1000320c` is inside `stdio_uart_out_flush` :

```
1000320c: 6993   ldr  r3, [r2, #24]
1000320e: 071b   lsls r3, r3, #28
10003210: d4fc   bmi.n 0x1000320c
```

That is the UART transmit-FIFO drain loop inside `printf`, so the core is running `main`'s loop and simply spends nearly all its time there. The breakpoint at `main` did not fire because `main`'s entry (`0x10000234`) runs exactly **once per reset**. If you arm the breakpoint after the reset, or set it while the target is already running and just resume, the core is already past `0x10000234` and will never re-execute it. Either arm the breakpoint **before** resetting, or break inside the loop at `0x1000023e`, which fires every iteration.

`0x1000320c` **is not a function.** It is one instruction inside `stdio_uart_out_flush`, which starts at `0x10003208`:

```
10003208 <stdio_uart_out_flush>:
10003208: 4b02   ldr  r3, [pc, #8]  ; @ 0x10003214
1000320a: 681a   ldr  r2, [r3]
1000320c: 6993   ldr  r3, [r2, #24] ; the core sits here while the UART drains
1000320e: 071b   lsls r3, r3, #28
10003210: d4fc   bmi.n 0x1000320c
10003212: 4770   bx lr
10003214: 20000850 .word 0x20000850
```

If Binary Ninja has created a function at `0x1000320c` (for example because the debugger stopped at that PC), the decompiler shows garbage: registers named `entry_r4` / `entry_r5`, and stores to invented constants like `0x3a` and `0xfffffff6`. Delete that bogus function (right-click it -> `Delete Function`, or put the cursor on it and press `U` to undefine) and reanalyze. The real function is `stdio_uart_out_flush` at `0x10003208`.

The console floods with `Failed to read memory` or `SWD DPIDR` reconnects

Two causes:

1. **Core 1 is exposed.** The scripts must run with `USE_CORE=0`. Stop the server, confirm only `core0` is reported, restart, then restart Binary Ninja.
2. **Binary Ninja probed an unmapped address for a register preview.** When a register holds an arbitrary integer or uninitialized value (e.g. `0x69000000`), Binary Ninja tries to preview that memory address, triggering an OpenOCD SWD fault and debug-port reconnect (`SWD DPIDR 0x4c013477`). In the **Debugger Console**, run `source pico-gdb-memory.gdb`. This restricts GDB to valid Flash and SRAM, filtering invalid reads locally so OpenOCD's SWD connection stays completely stable.

`Connect to Remote Process` is greyed out and Pause does nothing

Binary Ninja is in a stale session, usually because the debug server restarted while attached. Quit and reopen Binary Ninja (or the `.bndb`) and connect again.

The decompiler still shows the old value after patching

Right-click the function and choose `Reanalyze`.

Project 2's LED does not light at all after patching

You patched only some of the three GPIO 16 sites. All three of `0x1000023a`, `0x10000244`, and `0x10000252` must change.

Fallback: do the dynamic steps with GDB (macOS 27)

If Binary Ninja's debugger crashes on attach on macOS 27 (see Troubleshooting), you can still do the live hack with the ARM GDB from the toolchain, against the same OpenOCD server. The addresses and register values are identical to the GUI steps.

Start the debug server (Step 10), then in a new terminal:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt:

```
set architecture armv8-m.main
target extended-remote :3333
hbreak *0x1000023e
continue
```

Do **not** run `monitor reset run` before `hbreak`. `0x1000023e` is inside `main`'s loop, so the breakpoint fires on the next iteration with no reset. If you reset first, the core runs `main` and you will not catch it.

GDB stops at the `printf` call. Confirm the value, change it, and let it run:

```
info registers pc r1  # pc = 0x1000023e, r1 = 0x2b
set $r1 = 0x46
stepi
continue
```

The serial monitor prints `age: 70` for the iteration you changed — the same temporary live hack as editing `r1` in the Binary Ninja Registers widget. When you are done, press `Ctrl-C`, then `detach` and `quit`.

If you specifically want to stop at `main` (`0x10000234`), remember its entry runs only once per reset, so the breakpoint must be armed *before* the reset:

```
monitor reset halt
hbreak *0x10000234
continue
```

If you instead set it while the target is running and just `continue`, you will "blow past" `main` and catch the core inside `printf` — in this build at `0x1000320c`, the `stdio_uart_out_flush` UART-drain loop.

Project 2 is the same with the other call site and value:

```
hbreak *0x1000024e
continue
info registers pc r1  # pc = 0x1000024e, r1 = 0
set $r1 = 0x42
stepi
```

`hbreak` sets a hardware breakpoint, which is required for read-only flash. It works from plain GDB because GDB sends the 2-byte length the Cortex-M33 comparators need. Binary Ninja's **GDB MI** adapter goes through the same GDB, so its GUI breakpoints work too; the old **GDB RSP** adapter was the one that sent a 1-byte length and could not set breakpoints here.

Glossary

Term	Definition
`.bss`	Section for uninitialized global variables; zeroed by startup code
`.data`	Section for initialized global variables; copied from flash to SRAM at boot
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
`.rodata`	Read-only section for constants and string literals; stays in flash
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
Hardware breakpoint	A breakpoint serviced by the CPU comparators, required for read-only flash
Inlining	The optimizer replacing a function call with the function body; why `blink_and_print` disappears in `Release`
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`
MMIO	Memory-mapped I/O — hardware registers accessed as memory addresses
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at `0xd0000000`
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Prove the behavior dynamically, resolve the names from the ELF, then patch the bytes and flash.